

RESEARCH ARTICLE

Integrated Task Scheduling, Path Planning, and Control for Cooperative UGV-UAV Systems via an Extended MPPI-GA Framework

DA-HYUN NAM^{ID} AND HYEONG-GEUN KIM^{ID}

Department of Mechanical Engineering, Incheon National University, Incheon 22012, Republic of Korea

Corresponding author: Hyeong-Geun Kim (hgkim@inu.ac.kr)

This work was supported by Korean Government Korea AeroSpace Administration (KASA) under Grant RS-2022-NR067080.

ABSTRACT This study presents a cooperative algorithm for heterogeneous unmanned ground and aerial vehicles (UGVs and UAVs). Such cooperative systems have recently garnered significant attention in domains like disaster response, target tracking, and exploration. However, many existing methods are constrained by hierarchical structures for task assignment, path planning, and control, which limits adaptability in dynamic environments. Furthermore, these approaches often rely on clustering processes, with performance frequently sensitive to the chosen granularity. To address these limitations, this work extends the previously proposed model predictive path integral control-genetic algorithm (MPPI-GA) framework, initially developed for UGVs, to quadrotor UAVs, resulting in an output-sampled MPPI-GA (oMPPI-GA), by adapting it to their dynamic characteristics. Building upon this extension, a unified algorithm is proposed that integrates task scheduling, obstacle-avoidance path planning, and control input generation into a single, cohesive framework. The algorithm adopts an output-sampled MPPI structure to alleviate control input sampling sensitivity for UAVs, and a novel triple-layer chromosome enabling UAVs to generate closed tours without explicit task clustering. In this system, UAVs equipped with oMPPI-GA and UGVs using MPPI-GA cooperatively execute missions. A distributed task conflict resolution scheme facilitates seamless task allocation among these heterogeneous agents. The effectiveness of the proposed algorithm is validated through numerical simulations and real-world hardware experiments.

INDEX TERMS Cooperative systems, decentralized multi-agent systems, genetic algorithm, model predictive path integral control (MPPI), MPPI-GA, path planning, task assignment, triple-layer chromosome, UAV, UGV.

I. INTRODUCTION

Recently, cooperative systems involving unmanned aerial vehicles (UAVs) and unmanned ground vehicles (UGVs) have attracted significant attention. These systems are widely applied in domains such as exploration, target tracking, autonomous delivery, reconnaissance, and surveillance. UGVs are well-suited for long-range missions but are constrained by terrain and geographical limitations. In contrast, UAVs can hover vertically and move omnidirectionally, offering flexibility in accessing areas that are challenging for ground vehicles. However, their limited battery life imposes constraints on sustained operation. Consequently,

UAVs and UGVs are considered complementary platforms. This complementary nature makes them ideal for synergistic operations, where UGVs often handle broad-area ground exploration while UAVs conduct agile aerial missions, thereby maximizing overall mission effectiveness.

Most existing studies on cooperative UAV-UGV systems are characterized by a dominant agent, where either the UAV or the UGV takes the lead in mission execution. In UAV-dominant configurations, UAVs mainly perform aerial missions, while UGVs act as mobile charging or transport platforms. [1] introduces a two-level memetic algorithm that clusters aerial tasks and assigns rendezvous points for UGV-assisted recharging and relocation of UAVs. A related clustering and routing-based framework [2] deter-

The associate editor coordinating the review of this manuscript and approving it for publication was Zhenbao Liu^{ID}.

mines recharging depots and jointly optimizes UGV routes and UAV task sequences through the traveling salesperson problem (TSP) and vehicle routing problem (VRP) formulations. Similarly, generalized traveling salesperson problem (GTSP)-based coordination approaches [3], [4] plan UAV paths first and then derive UGV routes according to charging demands. From a complementary perspective, [5] initiates planning from the UGV side, evaluating UAV mission feasibility under refueling and charging constraints. Finally, [6] addresses a surveillance mission supported by a UGV along a predefined route, employing a hybrid estimation-of-distribution algorithm–genetic Algorithm (EDA–GA) for 3D trajectory planning, though its grid-based design limits scalability. Heuristic optimization methods are also utilized in related multi-UAV cooperative task assignment problems, such as [7], which develops an adaptive self-motivated teaching–learning-based optimization (AMTLBO) algorithm to address dual heterogeneities among UAVs and carrier platforms.

Other studies have explored UGV-dominant frameworks [8], [9], in which the UGV serves as the primary task executor and the UAV provides auxiliary perception or delivery support.

In non-dominant cooperative systems, both UGVs and UAVs actively execute their respective tasks while complementing each other's capabilities, with UAVs providing wide-area and terrain-adaptive coverage and rapid mobility, and UGVs offering endurance and ground-level support. Mission planning under communication constraints is addressed in [10], where an MILP formulation is solved using a branch-and-cut algorithm to obtain optimal solutions. A hybrid genetic algorithm is introduced in [11] to allocate tasks between UGVs and UAVs in a distributed manner. Reference [12] assigns exploration responsibilities to both platforms while also utilizing the UGV as a mobile charging asset; the authors combine classical combinatorial techniques with evolutionary methods to construct their strategy. Finally, [13] proposes a quantum-classical hybrid algorithm that merges quantum evolutionary search with traditional genetic algorithms, addressing mission allocation across heterogeneous agents including UAVs, UGVs, and unmanned surface vehicles (USVs).

While some approaches have explored distributed task allocation, many of the aforementioned studies predominantly rely on centralized or hierarchical architectures for overall mission planning and control [14]. In practice, however, such hierarchical frameworks can limit real-time adaptability and mission execution efficiency due to sequential processing and rigid task structures. Furthermore, many existing algorithms assign tasks assuming a 2D environment, even though UAVs operate in three-dimensional space during mission execution.

Recently, decentralized cooperative algorithms have also been developed. For instance, [15] proposes a graph neural operator-based approach that utilizes a graph attention task allocator for optimized target detection. However, in this

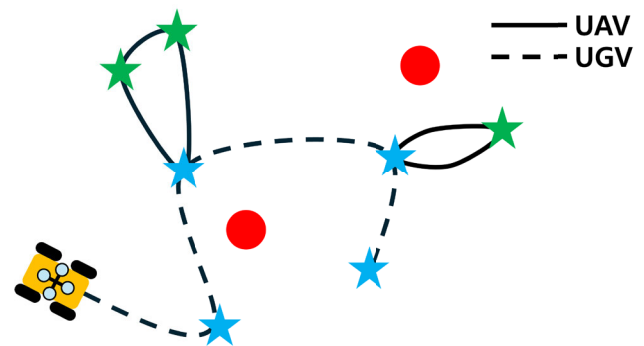


FIGURE 1. Example of the proposed problem.

method, task assignment and path planning are treated separately, and exploration is conducted on a 2D grid map. Another line of work [16] introduces a decentralized policy learning framework based on reinforcement learning. Although this study employs a constrained flashforward mechanism to enhance coordination, the agents considered are homogeneous UAVs with differing roles, rather than physically heterogeneous platforms like UAVs and UGVs. Such methods usually require extensive offline training, and their performance may degrade when deployed in environments that differ significantly from the training distribution.

In this study, we aim to enable both UGVs and UAVs to perform their respective tasks effectively: the UGV undertakes ground-level tasks, and the UAV handles aerial tasks, with all operations considered in a three-dimensional space. Rather than relying on a hierarchical architecture, we adopt an integrated algorithm that simultaneously performs task scheduling, obstacle-avoidance path planning, and plant-level control to enhance real-time responsiveness. To this end, we build upon our previously developed MPPI-GA algorithm [17], an integrated framework capable of handling these aspects within a single algorithmic loop. Compared with hierarchical architectures, MPPI-GA is better suited to adapting to dynamic mission scenarios and efficiently generating obstacle-avoiding trajectories. Furthermore, the generalized MPPI-GA naturally extends to multi-agent systems in a decentralized manner, ensuring scalability while preserving real-time responsiveness.

However, the original MPPI-GA was limited to planar UGV operations and therefore could not be directly applied to aerial platforms operating in three-dimensional space. In this work, we extend and adapt MPPI-GA for quadrotor UAVs, resulting in an output-sampled MPPI-GA (oMPPI-GA), by tailoring it to their specific dynamic characteristics. Building upon this extension, we further propose a cooperative algorithm that facilitates seamless collaboration between UAVs and UGVs in heterogeneous multi-agent systems. In this cooperative algorithm, the UGV follows an open tour to complete its assigned ground tasks, with each ground task location serving as a candidate takeoff and landing point for the UAV. From these candidate locations, the UAV performs

selected aerial tasks and returns to the same point, forming a closed tour, as illustrated in Figure 1.

Although quadrotor UAVs appear to move omnidirectionally, their translational accelerations are inherently coupled with roll–pitch attitude changes and bounded thrust, resulting in nonholonomic-like dynamic constraints. Such characteristics must be considered even in large-scale 3D environments, where long-distance flight accumulates the effects of limited acceleration, attitude transitions, and disturbances. Therefore, incorporating UAV dynamics is essential to ensure that the planned aerial trajectories remain feasible and reliable for real execution.

To summarize, the main contributions of this paper are as follows: 1. We propose an extended version of MPPI-GA applicable to quadrotor UAVs (oMPPI-GA). 2. We establish a cooperative planning framework combining oMPPI-GA for UAVs and MPPI-GA for UGVs in a decentralized setting. 3. The proposed method explicitly considers mission execution in three-dimensional environments. 4. The UAV can generate and execute its aerial task sequence without requiring a separate clustering algorithm, facilitated by the proposed triple-layer chromosome structure. 5. We propose a three-dimensional output sampling region for oMPPI, enabling its robust application to quadrotor UAVs.

Section II introduces the background of MPPI-GA and the system dynamics of a UGV and a UAV. Section III describes the oMPPI-GA algorithm tailored for quadrotors. Section IV outlines the integrated cooperative framework that facilitates interaction between the UGV and UAV. Section V demonstrates the effectiveness of the proposed approach through Python-based simulations and real-world hardware experiments, accompanied by a discussion of the outcomes. Finally, Section VI concludes the paper.

II. PRELIMINARIES

Before presenting our algorithms, this section briefly reviews essential background concepts. Subsection II-A provides an overview of the MPPI-GA algorithm. Subsections II-B and II-C then describe the dynamic models of the UGV and UAV, respectively, employed in this study.

A. MPPI-GA

The TSP is widely recognized as an NP-hard problem [18], [19]. Consequently, heuristic approaches such as genetic algorithm (GA) have been extensively employed to find near-optimal solutions [20], [21], [22]. We have recently proposed MPPI-GA, which integrates model predictive path integral control (MPPI) with a GA by leveraging their structural similarities [17]. MPPI-GA synergistically combines the sampling-based optimization capability of MPPI with the evolutionary search mechanism of a GA. MPPI is particularly well-suited for handling nonconvex cost functions and nonlinear system dynamics [23], [24], while GA enables efficient exploration in discrete task allocation scenarios. This unified algorithm performs task scheduling, obstacle-avoidance path planning, and control input genera-

tion, all within a single iterative loop. For further algorithmic details and its formal formulation, readers are referred to our previous work.

B. UNMANNED GROUND VEHICLE STATE AND DYNAMICS

The state of the UGV is characterized by its planar position (x, y) and heading angle ψ . Its kinematic model is given by:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \cos \psi & 0 \\ \sin \psi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (1)$$

The terms v and ω correspond to the UGV's linear and angular velocities, respectively, which serve as the control inputs.

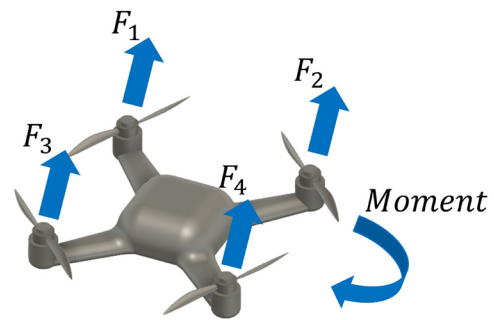


FIGURE 2. The configuration of the quadrotor.

C. UNMANNED AERIAL VEHICLE STATE AND DYNAMICS

A quadrotor is used as the aerial platform in this work, as depicted in Figure 2. The quadrotor is assumed to be symmetric with respect to its x - and y -axes, with the gravitational force acting at its center of mass. The thrust generated by each of the four rotors is denoted by F_i ($i = 1, 2, 3, 4$). The control inputs for the quadrotor, derived from these rotor forces, are defined as:

$$\begin{aligned} u_1 &= (F_1 + F_2 + F_3 + F_4)/m \\ u_2 &= (F_2 - F_4)/J_1 \\ u_3 &= (-F_1 + F_3)/J_2 \\ u_4 &= \rho(F_1 - F_2 + F_3 - F_4)/J_3 \end{aligned} \quad (2)$$

where m is the mass of the quadrotor, l is the distance from the center of mass to each rotor, J_1, J_2, J_3 are the moments of inertia about the principal body axes (roll, pitch, and yaw, respectively), and ρ is the force-to-moment scaling factor.

The dynamics of the quadrotor are then described by:

$$\begin{aligned} \ddot{x} &= u_1(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ \ddot{y} &= u_1(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ \ddot{z} &= u_1(\cos \phi \cos \theta) - g \\ \ddot{\phi} &= u_2/l \\ \ddot{\theta} &= u_3/l \\ \ddot{\psi} &= u_4 \end{aligned} \quad (3)$$

In this model, (x, y, z) are the inertial frame coordinates of the quadrotor's center of mass, and (ϕ, θ, ψ) are its Euler angles. The term g denotes gravitational acceleration. Equation (3) indicates that the total thrust normalized by mass, denoted by u_1 , governs translational acceleration. Meanwhile, u_2, u_3 , and u_4 are capable of generating angular accelerations for roll, pitch, and yaw, respectively.

III. oMPPI-GA

MPPI is known to be highly sensitive to the choice of mean and covariance for control input samples. While this issue is relatively manageable for UGV, which typically has two control inputs and a relative degree of one, it becomes significantly more challenging for UAVs. UAVs generally possess four control inputs and a relative degree of two, which makes it difficult to apply MPPI directly. As MPPI-GA is built upon MPPI, it also faces similar challenges.

To address this sensitivity to control input sampling, oMPPI was previously proposed [25]. However, the original oMPPI formulation was based on planar UGV dynamics and is not directly applicable to UAVs operating in three-dimensional space. Therefore, in this work, we modify the oMPPI framework to accommodate the specific dynamics of a quadrotor UAV. Furthermore, to enable the quadrotor to effectively plan and execute closed-tour missions involving multiple aerial tasks, modifications to the GA component are also necessary. Instead of a conventional single-layer chromosome representation, we introduce a novel GA structure incorporating a triple-layer chromosome. This structure enables the UAV to construct multiple closed tours by selecting and sequencing aerial tasks from candidate starting points, each defined by the UGV's ground task locations.

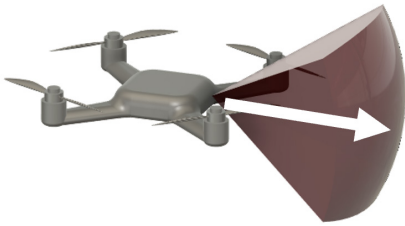


FIGURE 3. Sampling range of control inputs for the quadrotor.

A. oMPPI FOR QUADROTORS

The core idea of oMPPI is to sample candidate outputs instead of control inputs, subsequently computing the corresponding control inputs through inverse dynamics or other control methods. To apply and adapt this concept effectively to a quadrotor operating in three-dimensional space, we propose the use of a spherical pyramidal wedge as the output sampling region, illustrated in Figure 3. The radius r of this spherical sampling region is determined by the maximum velocity of the quadrotor, V_{max} , and the prediction horizon, H , as:

$$r = V_{max} \cdot H \quad (4)$$

The position of the quadrotor can be controlled using its roll angle ϕ , pitch angle θ , yaw rate $\dot{\psi}$, and total thrust T [26]. Reflecting this, the translational dynamics from Equation (3) can be reformulated in terms of total thrust T as:

$$\begin{aligned} \ddot{x} &= T(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi)/m \\ \ddot{y} &= T(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi)/m \\ \ddot{z} &= T(\cos \phi \cos \theta)/m - g \end{aligned} \quad (5)$$

We then define virtual control inputs v_1, v_2 , and v_3 as:

$$\begin{aligned} v_1 &= T \sin \phi / m \\ v_2 &= T \sin \theta \cos \phi / m \\ v_3 &= T \cos \phi \cos \theta / m \end{aligned} \quad (6)$$

Using these virtual inputs, Equation (5) can be expressed more compactly as:

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} \sin \psi & \cos \psi & 0 \\ -\cos \psi & \sin \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} \quad (7)$$

Instead of relying on an inverse model as proposed in the original oMPPI work, which may require additional model training or estimation, we compute the virtual control inputs using discrete sliding mode control (DSMC). To apply DSMC, Equation (5) is first transformed into a discrete-time system using the linear extrapolation method [27], yielding:

$$\begin{aligned} \begin{bmatrix} x_{k+2} \\ y_{k+2} \\ z_{k+2} \end{bmatrix} &= \begin{bmatrix} 2x_{k+1} - x_k \\ 2y_{k+1} - y_k \\ 2z_{k+1} - z_k - \Delta t^2 g \end{bmatrix} \\ &+ \Delta t^2 \begin{bmatrix} \sin \psi_k & \cos \psi_k & 0 \\ -\cos \psi_k & \sin \psi_k & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \end{aligned} \quad (8)$$

For position control using these virtual inputs, the sliding surface at time step k is defined as:

$$S(k) = \begin{bmatrix} a_x(x_k - x_k^d) + (dx_k - dx_k^d) \\ a_y(y_k - y_k^d) + (dy_k - dy_k^d) \\ a_z(z_k - z_k^d) + (dz_k - dz_k^d) \end{bmatrix} \quad (9)$$

where the coefficients a_x, a_y, a_z are positive constants, and the discrete derivative operator is defined as $dq_k = (q_{k+1} - q_k)/\Delta t$ for $q \in x, y, z$.

The updated sliding surface $S(k+1)$ is consequently:

$$S(k+1) = \begin{bmatrix} a_x(x_{k+1} - x_{k+1}^d) + (dx_{k+1} - dx_{k+1}^d) \\ a_y(y_{k+1} - y_{k+1}^d) + (dy_{k+1} - dy_{k+1}^d) \\ a_z(z_{k+1} - z_{k+1}^d) + (dz_{k+1} - dz_{k+1}^d) \end{bmatrix} \quad (10)$$

Employing the reaching law $S(k+1) = S(k) - q\Delta t \cdot S(k) - \eta\Delta t \cdot \text{sgn}(S(k))$ with $q > 0$, $\eta > 0$, and $0 < 1 - q\Delta t < 1$ [28], and substituting Equations (8), (9), and (10), the virtual control inputs at time step k are obtained as:

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \frac{1}{\Delta t} \begin{bmatrix} \sin \psi_k & \cos \psi_k & 0 \\ -\cos \psi_k & \sin \psi_k & 0 \\ 0 & 0 & 1 \end{bmatrix}^{-1}$$

$$\times \left(\begin{bmatrix} -dx_k + dx_{k+1}^d - a_x(x_{k+1} - x_{k+1}^d) \\ -dy_k + dy_{k+1}^d - a_y(y_{k+1} - y_{k+1}^d) \\ -dz_k + dz_{k+1}^d - a_z(z_{k+1} - z_{k+1}^d) + g\Delta t \\ +(1 - q\Delta t) \cdot S(k) - \eta\Delta t \cdot \text{sgn}(S(k)) \end{bmatrix} \right) \quad (11)$$

Finally, the attitude commands and total thrust are derived from the computed virtual control inputs as follows:

$$\begin{aligned} \phi_k &= \arctan(u_1 / \sqrt{u_2^2 + u_3^2}) \\ \theta_k &= \arctan(u_2 / u_3) \\ T_k &= m \cdot \sqrt{u_1^2 + u_2^2 + u_3^2} \end{aligned} \quad (12)$$

$$\dot{\psi}_k = K_\psi(\psi_k^d - \psi_k) \quad (13)$$

The yaw rate follows the expression in Equation (13) [29], where K_ψ is a positive constant.

B. TRIPLE-LAYER CHROMOSOME FOR UAV TASK SEQUENCING

To enable the UAV to generate routes involving multiple closed tours originating from and returning to UGV task locations, we propose a GA characterized by a triple-layer chromosome structure, as illustrated in Figure 4. In this chromosome representation, ground tasks are denoted by positive integers, while aerial tasks are represented by negative integers. The first layer encodes the sequence of ground tasks visited by the UGV that will serve as UAV deployment locations. The second layer specifies the number of aerial tasks that the UAV will perform during a sortie launched from each corresponding ground task in L1; the sum of these values equals the total number of pending aerial tasks. The third layer defines the execution sequence for the set of aerial tasks assigned to each sortie. Similar to the original MPPI-GA, this GA employs a parthenogenetic reproduction mechanism for the second and third layers. Partheno-reproduction [30] is adopted because it effectively preserves task selection while allowing flexible rearrangement of the task sequence within each chromosome. The evolutionary operations are randomly performed on each chromosome in the second and third layers during the reproduction phase and are described as follows:

$$\begin{aligned} \text{Swap} : [x_1 x_2 x_3 x_4 x_5 x_6] &\xrightarrow{x_1 \text{ and } x_4} [x_4 x_2 x_3 x_1 x_5 x_6] \\ \text{Slide} : [x_1 x_2 x_3 x_4 x_5 x_6] &\xrightarrow{[x_1 x_2 x_3]} [x_3 x_1 x_2 x_4 x_5 x_6] \\ \text{Flip} : [x_1 x_2 x_3 x_4 x_5 x_6] &\xrightarrow{[x_2 x_3 x_4 x_5]} [x_1 x_5 x_4 x_3 x_2 x_6] \end{aligned}$$

For instance, referring to Figure 4, if L1 indicates the UGV proceeds to ground task '3', and L2 specifies one aerial task from this point, L3 might list aerial task '-5'. The UAV would then launch from ground task '3', complete task '-5', return to '3', and rendezvous with the UGV. The UGV might then proceed to ground tasks '2', '4', and '7'. If, from ground task '7', L2 indicates three aerial tasks, L3 would provide their sequence, e.g., '-2', '-1', '-3', which the UAV executes before returning to task '7'. This process is repeated for other designated ground tasks like '9'.

L1	6	3	2	4	7	8	9	5	10	1
L2		1			3		2			
L3	-5	-2	-1	-3	-6	-4				

FIGURE 4. Triple-layer chromosome.

C. COST FUNCTION

For each sampled trajectory and task sequence, we define a cost function $q(\mathbf{x}, \text{seq})$, where $\mathbf{x}$ is the current state of the UAV and seq represents the planned task sequence. To optimize the ordering of aerial tasks across multiple closed tours while concurrently planning the UAV's path, the cost function must consider both the distance between consecutive tasks within a tour and the distance from the UAV's current state to the first task of the upcoming tour segment.

Before cost calculation, the full UAV task sequence is decoded from the triple-layer chromosome. Let $c_{a,b}$ denote the gene at layer a and position b of the chromosome. For each UGV ground task $T_0^i = c_{1,b}$, where $c_{2,b} > 0$, indicating the i -th sortie from ground task $c_{1,b}$ involving $c_{2,b}$ aerial tasks, the sequence of aerial tasks S^i is extracted from c_3 . This forms a closed tour for the UAV: $T^i = (T_0^i, S^i, T_0^i)$. The overall cost function is then:

$$\begin{aligned} T_0^i &= c_{1,b} \quad \text{where } c_{2,b} > 0 \\ V &= (c_{3,b})_{b \in B} \quad \text{where } B := \{b \mid c_{3,b} < 0\} \\ S^i &= (V_m)_{m=\sum_{p=1}^{i-1} c_p}^{\sum_{p=1}^i c_p} \\ T^i &= (T_0^i, S^i, T_0^i) \\ q(\mathbf{x}, \text{seq}) &= \frac{1}{2} d(\mathbf{x}, T_f)^2 + \frac{1}{2} \sum_{i=1}^n \sum_{j=0}^m d(T_j^i, T_{j+1}^i)^2 + k_c C \\ C &= \begin{cases} 1 & \text{if distance} < \text{tol} \\ 0 & \text{otherwise} \end{cases} \end{aligned} \quad (14)$$

Here, T_f is the first task in the immediate sequence for the UAV. The term $d(T_j^i, T_{j+1}^i)$ represents the distance between consecutive waypoints in the i -th tour. C is a binary collision penalty: $C = 1$ if the distance to the nearest obstacle is less than a predefined tolerance, and $C = 0$ otherwise, ensuring collision-free operation. The coefficient k_c is the penalty weight for collision violations.

The cost function in (14) corresponds to the running cost component of the oMPPI-GA rollout. The complete objective follows the standard MPPI structure including terminal and input energy costs. Therefore, no separate path-planning algorithm is used for cost evaluation, ensuring consistency between GA fitness evaluation and the actual MPPI execution cost.

D. STRUCTURE OF oMPPI-GA

Algorithm 1 summarizes the overall oMPPI-GA procedure implemented for the UAV. This algorithm integrates several key extensions to the original MPPI-GA framework, notably the incorporation of the oMPPI technique, using output

Algorithm 1 : oMPPI-GA

```

1:  $K$ : Number of samples
2:  $N$ : Number of time steps
3:  $\mathbf{x}_{sys}$ : Current system states
4:  $\Delta t$ : Sampling time
5:  $\lambda$ : Temperature parameter
6:  $r, \alpha, \beta$ : Radius, azimuth angle range, and inclination
   angle range of output candidates
7:  $P = (seq_0, seq_1, \dots, seq_{k-1})$ : Population set
8: Initialize  $P$ 
9: while task not completed do
10:   $P \leftarrow$  Genetic operator( $P$ )
11:  for  $k \leftarrow 0$  to  $K - 1$  do
12:     $\mathbf{x} \leftarrow \mathbf{x}_{sys}, S(k, 0) \leftarrow 0$ 
13:    [Sampling] Sample the  $k$ th trajectory rollout  $\delta Y(k)$ 

14:    for  $i \leftarrow 0$  to  $N - 1$  do
15:      Calculate the trajectory cost  $S(k, i)$ 
16:    end for
17:    end for
18:    for  $i \leftarrow 0$  to  $N - 1$  do
19:       $Y_i \leftarrow Y_i + \frac{\sum_{k=0}^{K-1} \exp(-S(i,k)/\lambda) \delta Y_{i,k}}{\sum_{k=0}^{K-1} \exp(-S(i,k)/\lambda)}$ 
20:      [DSMC] Compute the control inputs at time step  $i$ 
         $u(i) = DSMC(Y_i)$ 
21:    end for
22:    [Task sequence] Obtain the final task sequence
         $seq = \frac{\sum_{k=0}^{K-1} \sum_{i=0}^{N-1} \exp(-S(i,k)/\lambda) \delta seq_k}{\sum_{k=0}^{K-1} \sum_{i=0}^{N-1} \exp(-S(i,k)/\lambda)}$ 
23:    send to actuators( $u_0, seq$ )
24:     $k \leftarrow k + 1$ 
25:  end while

```

sampling and DSMC for control, tailored for quadrotor dynamics, and the use of the triple-layer chromosome structure within the GA for sophisticated closed-tour task scheduling in a cooperative context.

IV. COOPERATIVE SYSTEM

The cooperative system leverages the distinct capabilities of the UGV and the UAV, orchestrated through their respective planning and control algorithms, MPPI-GA and oMPPI-GA. This section details the architecture of this cooperative framework and the mechanisms enabling their coordinated operation.

A. FRAMEWORK ARCHITECTURE AND AGENT ROLES

In the proposed heterogeneous multi-agent system, both the UGV and UAV operate with a significant degree of autonomy in their local decision-making processes. The UGV employs the MPPI-GA algorithm to schedule and navigate its assigned ground tasks, typically forming an open-tour trajectory across its operational area. These ground tasks, or waypoints along the UGV's path, subsequently serve as candidate launch and recovery sites for the UAV.

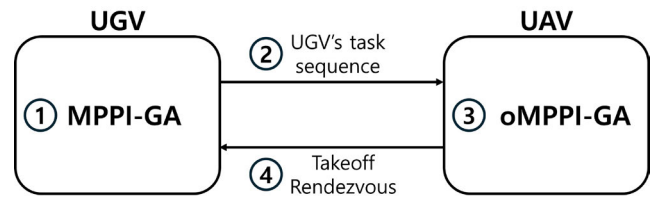


FIGURE 5. Structure of cooperative system.

The UAV, equipped with the oMPPI-GA algorithm, detailed in Section III, is responsible for executing aerial tasks. A key aspect of this framework is the UAV's ability to autonomously plan sequences of closed tours, originating from and returning to the UGV's designated task locations. This is facilitated by the triple-layer chromosome structure within its GA component, which allows the UAV to select optimal launch points from the UGV's planned route and determine the sequence of aerial tasks for each sortie without requiring pre-defined clustering of aerial tasks. This integrated approach, where task allocation for the UAV and path planning are unified within the oMPPI-GA loop, enhances the system's adaptability and efficiency. The overall system architecture thus promotes seamless aerial-ground coordination by design, minimizing the need for complex, separate coordination modules or rigid hierarchical planners.

B. DECENTRALIZED COORDINATION AND MISSION EXECUTION

The cooperative mission execution unfolds in a structured, step-by-step manner, exemplifying a decentralized coordination strategy. The process, illustrated in Figure 5, begins with the UGV planning its sequence of ground tasks using MPPI-GA. This planned task sequence is then transmitted to the UAV. Upon receiving this information, the UAV utilizes its oMPPI-GA algorithm to determine its own schedule of aerial tasks, planning closed tours that start and end at selected points from the UGV's received sequence. This sequential planning process, where the UGV's plan informs the UAV's planning constraints, serves as an inherent decentralized task allocation and conflict resolution mechanism; potential conflicts regarding UAV deployment locations and timing are resolved by the UGV's initial plan and the UAV's subsequent adaptive planning.

Physical coordination is achieved through explicit signals and predefined protocols. The UGV halts at designated ground task locations to allow for UAV takeoff or rendezvous upon completion of an aerial sortie. These actions are based on coordination signals exchanged between the two agents, ensuring safe and synchronized operations. This method allows the UAV to generate and execute its closed-tour aerial missions effectively without relying on conventional, often restrictive, task partitioning or clustering algorithms typically employed in hierarchical systems. The tight integration of planning and control within each agent's algorithm, combined with this clear coordination protocol, ensures real-time

responsiveness and simplifies the overall implementation of the cooperative system. The flexibility afforded by the UAV's oMPPI-GA, particularly its robust 3D operation and control input design decoupled from system-specific inverse models, further enhances the cooperative capabilities in complex environments.

V. EVALUATION

We validated the performance of our proposed cooperative algorithm through extensive numerical simulations and real-world hardware experiments. The simulations were conducted on a desktop computer equipped with an Intel Core i7-12700 @ 2.10GHz CPU and 32GB of RAM. All algorithms and simulation environments were implemented in Python.

A. SIMULATION

To evaluate the performance of the proposed algorithm, we compared it with adapted versions of two existing frameworks, ensuring consistency with our problem setting. The baseline frameworks are detailed as follows:

Framework 1-1 & 1-2: These are based on the two-level memetic algorithm presented in [1]. Framework 1-1 applies k-means clustering with two clusters, while Framework 1-2 uses three clusters to assess the impact of clustering granularity. In both versions, k-means clustering is performed prior to task scheduling; the UAV plans its tasks based on these clusters, and the UGV schedules its operations accordingly.

Framework 2: This framework adapts the TERRA algorithm from [12] to our setting. It uses Voronoi tessellation to cluster aerial tasks. In this approach, the UGV first schedules its task sequence, and the UAV subsequently plans its tasks based on the UGV's assignments.

As summarized in Table 1, the computational complexity of each framework is compared. Here, K denotes the number of sampled trajectories, N_g and N_a represent the numbers of ground and aerial tasks, respectively, H is the time horizon per sample, and s is the dimension of the state vector. For Framework 1, I_k and k indicate the number of iterations and clusters in the k -means process, while P_g, P_a and G_g, G_a denote the population sizes and generations of the memetic or GA-based optimizers for the UGV and UAV, respectively. d is the task feature dimension (typically $d = 2$ for planar coordinates), and $|C_i|$ is the number of aerial tasks in cluster i .

The proposed algorithm is a distributed, real-time framework; thus, the complexity $O(K(N_a + Hs))$ corresponds to the per-agent computation loop executed online. In contrast, Framework 1 and Framework 2 represent centralized or hierarchical batch processes, and their total computational cost depends heavily on the hyperparameters of the underlying optimizers (e.g., population size, number of generations, and clustering settings).

A key distinction in the visualizations, shown in Figures 6 – 9 and 10 – 13, is that our proposed algorithm

TABLE 1. Comparison of computational complexity.

Method	Computational complexity
Proposed algorithm	$O(K(N_a + Hs))$
Framework 1	$O(I_k k N_a d) + O(P_g G_g N_g^2) + O(P_a G_a \sum_i C_i ^2)$
Framework 2	$O(N_a \log N_a) + O(P_g G_g N_g^2) + O(P_a G_a N_a^2)$

generates trajectories that correspond to the actual paths executed by the UAV and UGV, as it integrates task scheduling, path planning, and control input generation. In contrast, the paths shown for the compared frameworks represent planned routes generated solely through their respective task scheduling mechanisms.

1) SCENARIO 1

This scenario was designed to evaluate the fundamental cooperative behavior of the UAV and UGV under a baseline mission environment without obstacles. It focuses on validating the integrated planning and control framework in a representative cooperative mission. This scenario involved four aerial tasks and six ground tasks. Figures 6–9 illustrate the resulting paths, where the blue solid line represents the UAV's trajectory, the orange solid line indicates the UGV's trajectory, and green star-shaped markers denote task locations. To provide a sense of spatial depth, the markers were rendered with varying color intensities according to their relative distance from the viewpoint (using the depthshade option).

As shown in Table 2, the proposed algorithm achieved a shorter total travel distance compared to the hierarchical frameworks. This performance gain arises from its integrated cooperative planning structure, wherein the UAV dynamically schedules its aerial tasks based on the sequence generated by the UGV. In the proposed framework, each agent independently computes its own mission assignment, path planning, and control input in real time. The UGV performs mission sequencing and path optimization for ground tasks, while the UAV forms multiple closed tours that adapt to the UGV's plan through online coordination.

Despite being an online algorithm, the proposed method imposes a relatively low computational burden. The computational complexity of each MPPI-GA and oMPPI-GA is $O(K(N_g + Hs))$ and $O(K(N_a + Hs))$, respectively. Owing to the distributed and parallelized computation structure, the proposed framework enables both agents to plan and control in real time while maintaining computational efficiency.

As illustrated in Figure 6, the proposed framework enables each agent to plan its own mission in real time while maintaining cooperative consistency. Despite being executed in an online manner, both the UAV and UGV generated the shortest and most compact trajectories among the compared methods, efficiently completing all assigned tasks without unnecessary detours or overlapping regions. These results verify that the proposed algorithm achieves effective and scalable cooperative task execution even in complex mission environments.

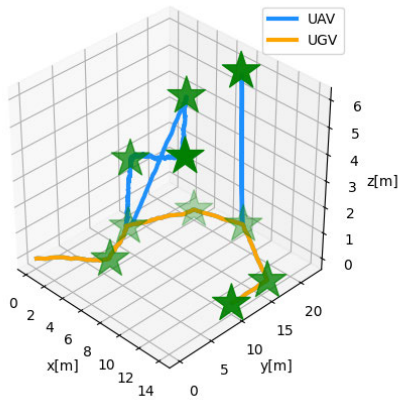


FIGURE 6. Simulation result using Framework 2 in Scenario 1.

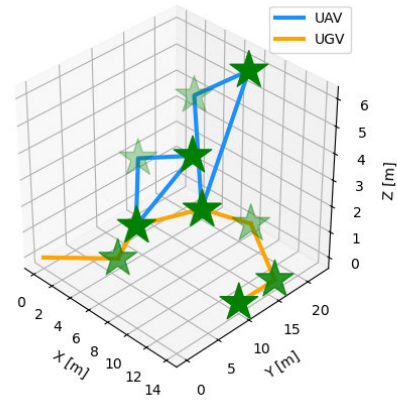


FIGURE 8. Simulation result using Framework 1-2 in Scenario 1.

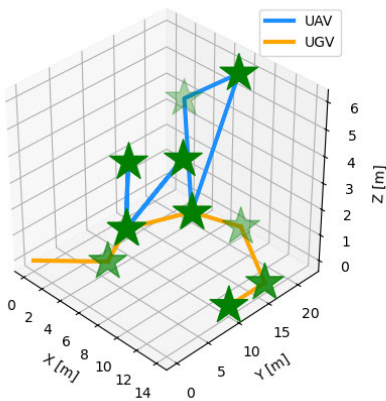


FIGURE 7. Simulation result using Framework 1-1 in Scenario 1.

TABLE 2. Comparison of the total distance obtained by each framework under Scenario 1.

Method	Distance [m]
Proposed algorithm	67.80
Framework 1-1	74.76
Framework 1-2	80.09
Framework 2	80.46

2) SCENARIO 2

To further verify the generality of our algorithm, simulations were conducted in a randomly generated mission scenario comprising five aerial tasks and ten ground tasks, which represents a more complex setup than Scenario 1. Figures 10–13 illustrate the trajectories obtained from each framework, while Table 3 summarizes their quantitative performance. Even with a total of 15 tasks, the proposed distributed framework maintained scalability and achieved shorter total travel distances compared to conventional hierarchical algorithms, demonstrating its superior efficiency in cooperative mission environments.

3) SCENARIO 3

To evaluate the performance of the proposed algorithm under a large-scale cooperative mission setting, additional simulations were conducted in Scenario 3. In this scenario,

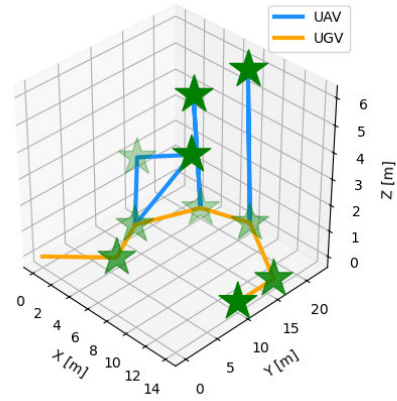


FIGURE 9. Simulation result using Framework 2 in Scenario 1.

TABLE 3. Comparison of total distance with different frameworks in Scenario 2.

Method	Distance [m]
Proposed algorithm	1273.87
Framework 1-1	1,485.10
Framework 1-2	1,511.69
Framework 2	1,433.26

a total of 50 tasks were randomly generated, consisting of 30 ground tasks and 20 aerial tasks. For Framework 1, the number of clusters was set to $k = 8$, following the original configuration of the clustering-based approach. Figures 14–16 present the resulting trajectories produced by each framework, and Table 4 provides a quantitative comparison of their performance.

Despite the significantly increased problem size, the proposed distributed framework consistently achieved shorter total travel distances than the conventional hierarchical methods. This result indicates that the proposed approach maintains scalability and efficiency even as the number of tasks increases, highlighting its suitability for large-scale cooperative UAV–UGV mission planning.

4) SCENARIO 4: OBSTACLE ENVIRONMENT

This scenario was designed to test the algorithm's capability in environments with obstacles. The proposed algorithm

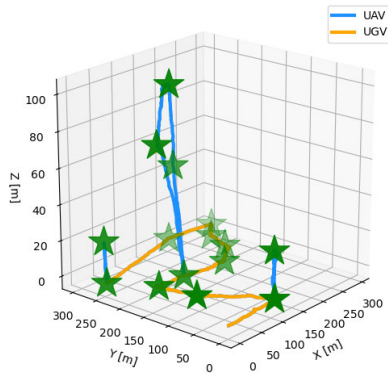


FIGURE 10. Simulation result using the proposed algorithm in Scenario 2.

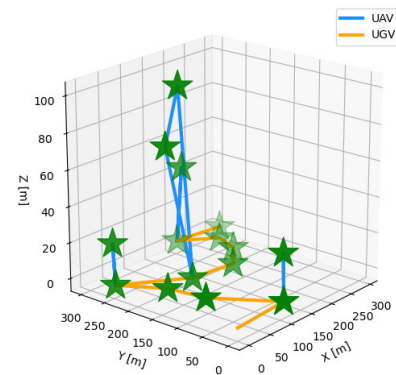


FIGURE 13. Simulation result using Framework 2 in Scenario 2.

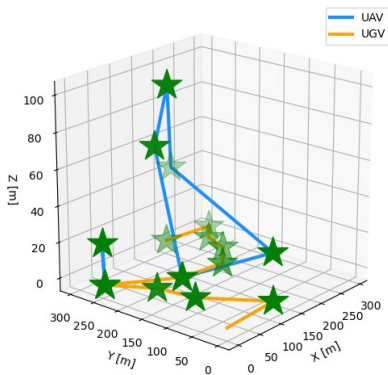


FIGURE 11. Simulation result using Framework 1-1 in Scenario 2.

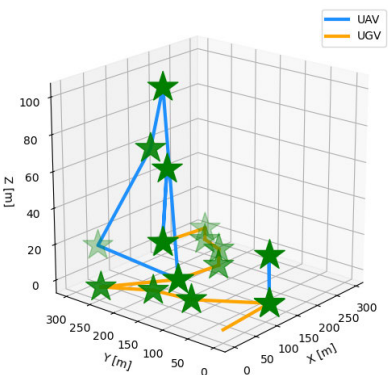


FIGURE 12. Simulation result using Framework 1-2 in Scenario 2.

directly generates obstacle-avoiding paths through its cost function, without requiring any dedicated path-adjustment modules for obstacle avoidance. Figure 17 presents the 3D simulation results from both top and side perspectives, where red cylinders represent obstacles. To ensure safer avoidance, a safety margin was applied around each obstacle in the cost function, allowing the UAV to maintain an appropriate distance from the cylindrical obstacles during flight. As illustrated in Figure 17, the UAV guided by our algorithm successfully navigated around obstacles while maintaining smooth and compact trajectories. These results confirm that the proposed algorithm can robustly generate

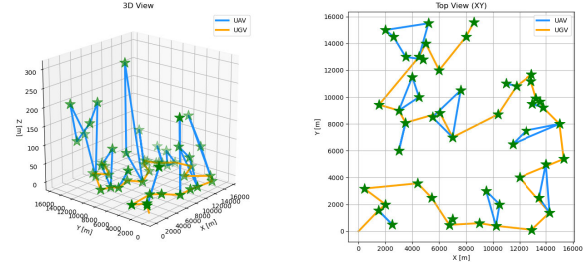


FIGURE 14. Simulation result using the proposed algorithm in Scenario 3.

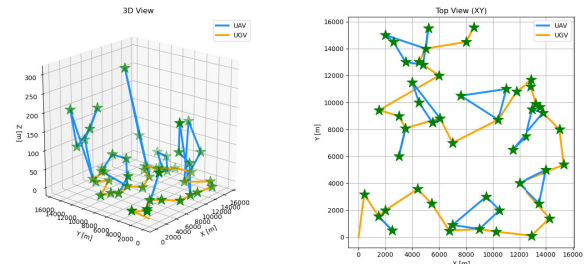


FIGURE 15. Simulation result using Framework 1 in Scenario 3.

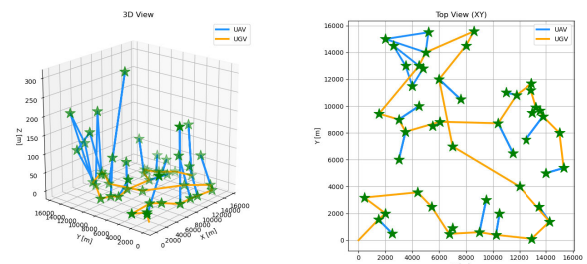


FIGURE 16. Simulation result using Framework 2 in Scenario 3.

collision-free paths by embedding obstacle penalties and safety margins directly into the cost function, demonstrating that the proposed method maintains consistent performance even in the presence of obstacles, despite the sampling-based nature of MPPI.

5) SCENARIO 5: MID-MISSION TASK ADDITION

This simulation demonstrated the robustness of the proposed algorithm against dynamic changes, specifically the addition

TABLE 4. Total travel distance comparison across different frameworks in Scenario 3.

Method	Distance [m]
Proposed algorithm	119,728.94
Framework 1	120,768.01
Framework 2	128,567.79

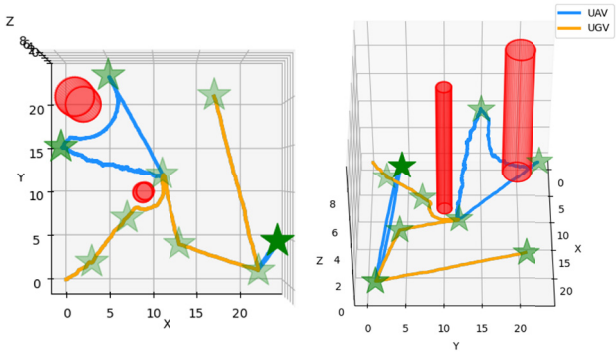


FIGURE 17. Simulation result in an environment with obstacles.

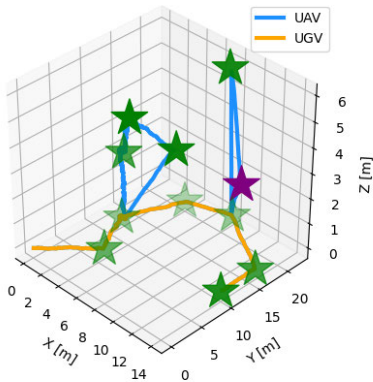


FIGURE 18. Simulation result under dynamic task insertion.



FIGURE 19. Our mobile robot used in the experiment.

of a new task during mission execution. Figure 18 illustrates this scenario, where a purple star-shaped marker indicates the newly introduced task. The UAV immediately responded to this new assignment, adjusting its path mid-mission to address the closer task first, showcasing its adaptability.

B. HARDWARE DEMONSTRATION

For real-world validation, experiments were conducted using the Scout-mini, a four-wheel-drive mobile robot, as the UGV and a Crazyflie 2.1 as the UAV. These robotic platforms are



FIGURE 20. Crazyflie 2.1 used in the experiment.

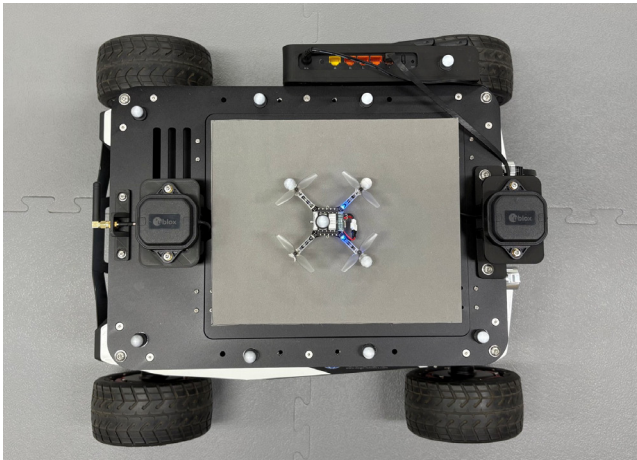


FIGURE 21. Experimental setup using a UGV and a UAV for cooperative task execution.

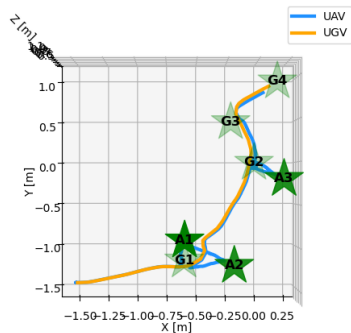


FIGURE 22. Top view of the first experimental result of the proposed algorithm.

depicted in Figure 19 and Figure 20, respectively. The UGV was equipped with an Intel NUC, featuring an Intel Core i7-1165G7 processor operating at 2.8 GHz and 16 GB of RAM, running Ubuntu 22.04. As shown in Figure 21, the UAV was carried by the UGV, taking off only to perform its assigned aerial tasks and subsequently returning to the UGV for rendezvous.

The experimental environment and operational setup are presented in Figures 22–24. The UAV executed the proposed oMPPI-GA algorithm on a separate desktop computer; computed control inputs were transmitted via ROS 2 and a Crazyradio PA. The UGV operated using its onboard computing unit, with both platforms performing real-time processing. Position sensing for both vehicles was achieved using an OptiTrack system, with its interface node developed

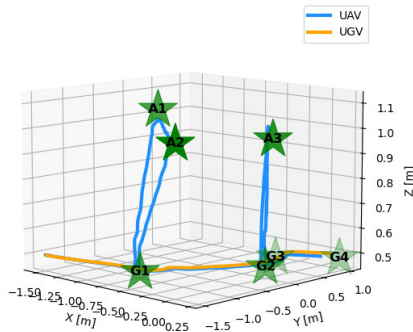


FIGURE 23. Side view of the first experimental result of the proposed algorithm.

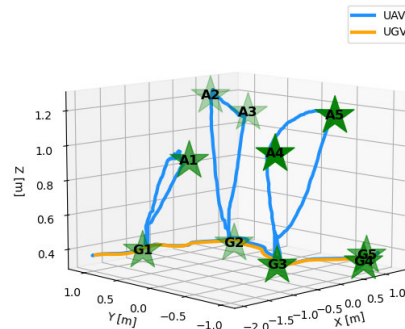


FIGURE 24. Second experimental result of the proposed algorithm.

in MATLAB R2024a. All other algorithms for control, planning, and coordination were implemented in Python and ROS 2. In the figure, task points are indicated by star-shaped markers. The tasks assigned to the UGV are labeled sequentially as $G1, G2, \dots, G_n$, representing the order of task execution, while those assigned to the UAV are denoted as $A1, A2, \dots, A_m$. In both hardware demonstrations, the cooperative UGV-UAV team successfully completed all assigned tasks while maintaining compact and efficient paths.

Notably, the UAV equipped with the proposed oMPPI-GA framework was able to perform appropriate task execution through real-time information exchange with the UGV, without requiring any additional clustering or task-partitioning algorithm.

VI. CONCLUSION

In this study, we introduced oMPPI-GA, an extension of our previously developed MPPI-GA algorithm, specifically tailored for quadrotor UAVs. This enhanced algorithm, oMPPI-GA, functions as an integrated solution that, in conjunction with MPPI-GA for UGVs, enables cooperative task scheduling, obstacle-avoidance path planning, and control input generation within heterogeneous UGV-UAV systems. The proposed oMPPI-GA framework addresses the sensitivity of traditional MPPI-based approaches to control input sampling by incorporating an output-sampled structure adapted to UAV dynamics. Furthermore, the introduction of a

novel triple-layer chromosome allows UAVs to dynamically plan and execute closed-tour aerial tasks based on the UGV's task sequence, thereby eliminating the need for pre-defined task clustering. Comprehensive numerical simulations and real-world hardware experiments verified that the cooperative framework achieves robust and efficient mission execution under various challenging conditions, including obstacle-rich environments and dynamic task insertions.

Future work will extend the proposed framework to more complex multi-agent settings involving multiple UAVs and UGVs with diverse roles, requiring more advanced task allocation and prioritization strategies. We also plan to investigate operation under communication constraints and partial observability to enhance decentralized coordination. In addition, incorporating learning-based components, such as reinforcement learning, to guide the sampling process within oMPPI-GA, may further improve adaptability in uncertain or dynamic environments.

REFERENCES

- [1] J. Li, T. Sun, X. Huang, L. Ma, Q. Lin, J. Chen, and V. C. M. Leung, "A memetic path planning algorithm for unmanned air/ground vehicle cooperative detection systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 4, pp. 2724–2737, Oct. 2022.
- [2] S. Ramasamy, J.-P.-F. Reddinger, J. M. Dotterweich, M. A. Childers, and P. A. Bhounsule, "Cooperative route planning of multiple fuel-constrained unmanned aerial vehicles with recharging on an unmanned ground vehicle," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2021, pp. 155–164.
- [3] K. Yu, A. K. Budhiraja, and P. Tokekar, "Algorithms for routing of unmanned aerial vehicles with mobile recharging stations," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 5720–5725.
- [4] Q. Wang, H. Chen, L. Qiao, J. Tian, and Y. Su, "Path planning for UAV/UGV collaborative systems in intelligent manufacturing," *IET Intell. Transp. Syst.*, vol. 14, no. 11, pp. 1475–1483, Nov. 2020.
- [5] M. S. Mondal, S. Ramasamy, J. D. Humann, J. M. Dotterweich, J.-P.-F. Reddinger, M. A. Childers, and P. A. Bhounsule, "Cooperative multi-agent planning framework for fuel constrained UAV-UGV routing problem," *J. Intell. Robot. Syst.*, vol. 111, no. 1, pp. 1–17, Jan. 2025.
- [6] Y. Wu, S. Wu, and X. Hu, "Cooperative path planning of UAVs & UGVs for a persistent surveillance task in urban environments," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4906–4919, Mar. 2021.
- [7] Y. Xinyong, L. Xin, W. Lei, J. Junhong, Z. Genlai, S. Xichao, T. Laifa, L. Chen, and W. Xinwei, "Carrier platform-enhanced multiple-UAV cooperative task assignment with dual heterogeneities," *Artif. Intell. Rev.*, vol. 58, no. 8, p. 248, May 2025.
- [8] W. Gao, J. Luo, W. Zhang, W. Yuan, and Z. Liao, "Commanding cooperative UGV-UAV with nested vehicle routing for emergency resource delivery," *IEEE Access*, vol. 8, pp. 215691–215704, 2020.
- [9] S. Kim and I. Moon, "Traveling salesman problem with a drone station," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 1, pp. 42–52, Jan. 2019.
- [10] S. G. Manyam, K. Sundar, and D. W. Casbeer, "Cooperative routing for an air-ground vehicle team—exact algorithm, transformation method, and heuristics," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 537–547, Jan. 2019.
- [11] L. Yu, Y. Yang, X. Su, S. Sun, T. Jiang, and J. Huang, "Cooperative task assignment for aerial-ground detection systems via a novel hybrid genetic method," *IEEE Trans. Ind. Electron.*, vol. 72, no. 4, pp. 4063–4072, Apr. 2025.
- [12] F. Ropero, P. Muñoz, and M. D. R-Moreno, "Terra: A path planning algorithm for cooperative UGV-UAV exploration," *Eng. Appl. Artif. Intell.*, vol. 78, pp. 260–272, Feb. 2019.
- [13] N. Furkan Bar, H. Yetiş, N. Özbey, and M. Karaköse, "Quantum computing based collaborative optimum mission allocation approach for heterogeneous multi unmanned vehicles," *IEEE Access*, vol. 12, pp. 169069–169078, 2024.

- [14] J. Jin, G. Zhang, X. Li, X. Su, C. Lu, Y. Cheng, Y. Ding, L. Wang, and X. Wang, "Cross-platform mission planning for UAVs under carrier delivery mode," *Defence Technol.*, vol. 53, pp. 76–97, Nov. 2025.
- [15] J. Peng, H. Viswanath, and A. Bera, "Graph-based decentralized task allocation for multi-robot target localization," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 10676–10683, Nov. 2024.
- [16] W. Dai, U. Rai, J. Chiun, Y. Cao, and G. Sartoretti, "Heterogeneous multi-robot task allocation and scheduling via reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2654–2661, Mar. 2025.
- [17] D.-H. Nam and H.-G. Kim, "MPPI-based integrated task assignment, path planning, and control for unmanned ground vehicles," *Int. J. Control, Autom. Syst.*, vol. 22, no. 12, pp. 3641–3652, Dec. 2024.
- [18] W. J. Cook, D. L. Applegate, R. E. Bixby, and V. Chvátal, *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ, USA: Princeton Univ. Press, 2011.
- [19] M. Held, A. J. Hoffman, E. L. Johnson, and P. Wolfe, "Aspects of the traveling salesman problem," *IBM J. Res. Develop.*, vol. 28, no. 4, pp. 476–486, Jul. 1984.
- [20] N. M. Razali and J. Geraghty, "Genetic algorithm performance with different selection strategies in solving tsp," in *Proc. World Congr. Eng.*, vol. 2, 2011, pp. 1–6.
- [21] S. Chatterjee, C. Carrera, and L. A. Lynch, "Genetic algorithms and traveling salesman problems," *Eur. J. Oper. Res.*, vol. 93, no. 3, pp. 490–510, 1996.
- [22] H. Braun, "On solving travelling salesman problems by genetic algorithms," in *Proc. Int. Conf. Parallel Problem Solving Nature*. Cham, Switzerland: Springer, 2006, pp. 129–133.
- [23] G. Williams, P. Drews, B. Goldfain, J. M. Reh, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2016, pp. 1433–1440.
- [24] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *J. Guid., Control, Dyn.*, vol. 40, no. 2, pp. 344–357, Feb. 2017.
- [25] L. L. Yan and S. Devasia, "Output-sampled model predictive path integral control (o-MPPI) for increased efficiency," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 14279–14285.
- [26] C. Montañez-Molina and J. Pliego-Jiménez, "Position and reduced attitude trajectory tracking control of quadrotors: Theory and experiments," *Aerosp. Sci. Technol.*, vol. 155, Dec. 2024, Art. no. 109683.
- [27] J.-J. Xiong and G. Zhang, "Discrete-time sliding mode control for a quadrotor UAV," *Optik*, vol. 127, no. 8, pp. 3718–3722, Apr. 2016.
- [28] W. Gao, Y. Wang, and A. Homaifa, "Discrete-time variable structure control systems," *IEEE Trans. Ind. Electron.*, vol. 42, no. 2, pp. 117–122, Feb. 1995.
- [29] Y. Okuyama, *Discrete Control Systems*. Cham, Switzerland: Springer, 2014.
- [30] Z. Wang, X. Fang, H. Li, and H. Jin, "An improved partheno-genetic algorithm with reproduction mechanism for the multiple traveling salesperson problem," *IEEE Access*, vol. 8, pp. 102607–102615, 2020.



DA-HYUN NAM received the B.S. degree in mechanical engineering from Incheon National University, in 2023. Her research interests include multi-agent systems, task assignment, path planning, and intelligent control.



HYEONG-GEUN KIM received the B.S., M.S., and Ph.D. degrees in mechanical and aerospace engineering from Seoul National University, in 2012, 2014, and 2018, respectively. He is currently an Associate Professor with the Department of Mechanical Engineering, Incheon National University. His research interests include guidance and control, nonlinear control, unmanned vehicles, intelligent control, and multi-agent systems.

...